

题目 1：空指针解引用（NULL）

源代码

```
#include <stdio.h>

int main(void)
{
    int *p = NULL;
    printf("p=%p\n", (void *)p);
    *p = 42;
    printf("unreachable\n");
    return 0;
}
```

直接运行现象

```
p=(nil)
exit:139
```

分析

该程序将指针 `p` 置为 `NULL`（地址 0），随后执行 `*p = 42`。这会向地址 0 写入一个 `int`（4 字节），属于空指针解引用引发的非法写内存。

CPU 触发保护异常，进程收到 `SIGSEGV` 并终止。本次运行的退出码为 139，通常对应 128+11，即被信号 11（`SIGSEGV`）杀死。

题目 2：使用 gdb 调试空指针解引用

源代码

```
#include <stdio.h>

int main(void)
{
    int *p = NULL;
    printf("p=%p\n", (void *)p);
    *p = 42;
    printf("unreachable\n");
    return 0;
}
```

gdb 输出

```
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
```

```
p=(nil)

Program received signal SIGSEGV, Segmentation fault.
0x0000555555555519c in main () at 02_null_gdb.c:7
7          *p = 42;
#0  0x0000555555555519c in main () at 02_null_gdb.c:7
#0  0x0000555555555519c in main () at 02_null_gdb.c:7
7          *p = 42;
p = 0x0
$1 = (int *) 0x0
```

分析

gdb 显示程序收到了 SIGSEGV，并停在 02_null_gdb.c:7 的 `*p = 42` 这一行。这说明崩溃点就是对空指针的写操作。回溯 (bt) 只有 main 一帧，符合该示例程序结构。同时 gdb 打印 `p = 0x0`，确认指针值为 0 (NULL)，因此对其解引用必然触发非法访问。

题目 3：使用 valgrind 分析空指针解引用

源代码

```
#include <stdio.h>

int main(void)
{
    int *p = NULL;
    printf("p=%p\n", (void *)p);
    *p = 42;
    printf("unreachable\n");
    return 0;
}
```

valgrind 输出

```
==138968== Memcheck, a memory error detector
==138968== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==138968== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==138968== Command: ./03_null_valgrind
==138968==
p=(nil)
==138968== Invalid write of size 4
==138968==    at 0x10919C: main (03_null_valgrind.c:7)
==138968==   Address 0x0 is not stack'd, malloc'd or (recently) free'd
==138968==
==138968==
```

```
==138968== Process terminating with default action of signal 11 (SIGSEGV)
==138968== Access not within mapped region at address 0x0
==138968== at 0x10919C: main (03_null_valgrind.c:7)
==138968== If you believe this happened as a result of a stack
==138968== overflow in your program's main thread (unlikely but
==138968== possible), you can try to increase the size of the
==138968== main thread stack using the --main-stacksize= flag.
==138968== The main thread stack size used in this run was 8388608.
==138968==
==138968== HEAP SUMMARY:
==138968== in use at exit: 1,024 bytes in 1 blocks
==138968== total heap usage: 1 allocs, 0 frees, 1,024 bytes allocated
==138968==
==138968== LEAK SUMMARY:
==138968== definitely lost: 0 bytes in 0 blocks
==138968== indirectly lost: 0 bytes in 0 blocks
==138968== possibly lost: 0 bytes in 0 blocks
==138968== still reachable: 1,024 bytes in 1 blocks
==138968== suppressed: 0 bytes in 0 blocks
==138968== Reachable blocks (those to which a pointer was found) are not shown.
==138968== To see them, rerun with: --leak-check=full --show-leak-kinds=all
==138968==
==138968== For lists of detected and suppressed errors, rerun with: -s
==138968== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
Segmentation fault (core dumped)
```

分析

valgrind 的 Invalid write of size 4 表示发生了一次非法写入，写入大小为 4 字节（与 int 大小一致）。报告定位到 03_null_valgrind.c:7，对应语句 `*p = 42`。

Address 0x0 is not stack'd, malloc'd or (recently) free'd 说明目标地址 0x0 不属于栈区、堆区（malloc/realloc 分配的区域），也不是刚释放的块，因此属于无效地址访问。

后续的 SIGSEGV 与 “Access not within mapped region at address 0x0” 进一步确认是对未映射地址的访问。此外 HEAP/LEAK SUMMARY 里的 still reachable 多为运行时库内部分配（例如 I/O 缓冲），关键问题仍是非法写导致崩溃。

题目 4：malloc 后忘记 free（内存泄漏）

源代码

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    size_t n = 1024;
```

```

    char *buf = (char *)malloc(n);
    if (buf == NULL)
    {
        perror("malloc");
        return 1;
    }

    buf[0] = 'A';
    buf[n - 1] = 'Z';
    printf("buf[0]=%c, buf[%zu]=%c\n", buf[0], n - 1, buf[n - 1]);

    return 0;
}

```

直接运行现象

```

buf[0]=A, buf[1023]=Z
exit:0

```

valgrind 输出

```

==139136== Memcheck, a memory error detector
==139136== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==139136== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==139136== Command: ./04_leak
==139136==
buf[0]=A, buf[1023]=Z
==139136==
==139136== HEAP SUMMARY:
==139136==      in use at exit: 1,024 bytes in 1 blocks
==139136==    total heap usage: 2 allocs, 1 frees, 2,048 bytes allocated
==139136==
==139136== 1,024 bytes in 1 blocks are definitely lost in loss record 1 of 1
==139136==    at 0x4846828: malloc (in
↳ /home/amadeus/.local/valgrind/usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==139136==    by 0x1091A8: main (04_leak.c:7)
==139136==
==139136== LEAK SUMMARY:
==139136==    definitely lost: 1,024 bytes in 1 blocks
==139136==    indirectly lost: 0 bytes in 0 blocks
==139136==    possibly lost: 0 bytes in 0 blocks
==139136==    still reachable: 0 bytes in 0 blocks
==139136==    suppressed: 0 bytes in 0 blocks
==139136==
==139136== For lists of detected and suppressed errors, rerun with: -s

```

```
==139136== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

分析

该程序分配了 1024 字节（`malloc(n)`）并正常使用，但退出前没有 `free(buf)`。由于进程结束时操作系统会回收该进程的整个虚拟地址空间，所以直接运行看起来“完全正常”。

`valgrind` 会在进程退出时做堆内存审计：

- `in use at exit`: 退出时仍未释放的堆内存。
- `definitely lost`: 确定泄漏。
- 本例显示 1,024 bytes ... `definitely lost`，并回溯到 `04_leak.c:7` 的 `malloc`。

结论：程序功能正确，但存在内存泄漏，需要在返回前添加 `free(buf)`。

题目 5：数组越界写（`data[100]`）

源代码

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *data = (int *)malloc(100 * sizeof(int));
    if (data == NULL)
    {
        perror("malloc");
        return 1;
    }

    for (int i = 0; i < 100; i++)
    {
        data[i] = i;
    }

    data[100] = 0;

    printf("data[0]=%d\n", data[0]);
    free(data);
    return 0;
}
```

直接运行现象

```
data[0]=0
exit:0
```

valgrind 输出

```
==139266== Memcheck, a memory error detector
==139266== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==139266== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==139266== Command: ./05_oob
==139266==
==139266== Invalid write of size 4
==139266==    at 0x109216: main (05_oob.c:18)
==139266==   Address 0x4a761d0 is 0 bytes after a block of size 400 alloc'd
==139266==    at 0x4846828: malloc (in
↳ /home/amadeus/.local/valgrind/usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==139266==   by 0x1091BE: main (05_oob.c:6)
==139266==
data[0]=0
==139266==
==139266== HEAP SUMMARY:
==139266==   in use at exit: 0 bytes in 0 blocks
==139266== total heap usage: 2 allocs, 2 frees, 1,424 bytes allocated
==139266==
==139266== All heap blocks were freed -- no leaks are possible
==139266==
==139266== For lists of detected and suppressed errors, rerun with: -s
==139266== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

分析

`malloc(100 * sizeof(int))` 仅分配了 100 个 `int` 的空间，有效下标范围为 0..99。语句 `data[100] = 0` 会写入数组末尾之后的 1 个元素位置，属于越界写。

直接运行时程序没有立即崩溃，可能因为越界写不一定立刻触发保护异常，但程序在逻辑上是不正确的。

valgrind 报告 `Invalid write of size 4`，并指出写入地址位于“0 bytes after a block of size 400 alloc'd”。这里 400 字节对应 100 个 `int`，说明越界写刚好发生在分配块的末尾之后，并定位到 `05_oob.c:18`。

题目 6：释放后继续使用（use-after-free）

源代码

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *data = (int *)malloc(100 * sizeof(int));
    if (data == NULL)
    {
        perror("malloc");
    }
}
```

```

        return 1;
    }

    for (int i = 0; i < 100; i++)
    {
        data[i] = i * 2;
    }

    free(data);

    printf("data[10]=%d\n", data[10]);

    return 0;
}

```

直接运行现象

```

data[10]=20
exit:0

```

valgrind 输出

```

==139365== Memcheck, a memory error detector
==139365== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==139365== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==139365== Command: ./06_uaf
==139365==
==139365== Invalid read of size 4
==139365==    at 0x109222: main (06_uaf.c:20)
==139365==   Address 0x4a76068 is 40 bytes inside a block of size 400 free'd
==139365==    at 0x484988F: free (in
↳ /home/amadeus/.local/valgrind/usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==139365==   by 0x109219: main (06_uaf.c:18)
==139365==   Block was alloc'd at
==139365==    at 0x4846828: malloc (in
↳ /home/amadeus/.local/valgrind/usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==139365==   by 0x1091BE: main (06_uaf.c:6)
==139365==
data[10]=20
==139365==
==139365== HEAP SUMMARY:
==139365==    in use at exit: 0 bytes in 0 blocks
==139365==   total heap usage: 2 allocs, 2 frees, 1,424 bytes allocated
==139365==
==139365== All heap blocks were freed -- no leaks are possible

```

```
==139365==
==139365== For lists of detected and suppressed errors, rerun with: -s
==139365== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

分析

在 `free(data)` 之后，`data` 指向的堆块已经被释放；继续读取 `data[10]` 属于释放后使用（Use-After-Free），是未定义行为。

该程序似乎仍然能跑，可能是释放后的堆块短时间内没有被分配器复用/覆盖，所以读到了旧内容。

`valgrind` 清楚给出 `Invalid read of size 4`，并指明该地址位于“已 free 的 400 字节块内部”，同时给出 `malloc/free` 的回溯，便于定位问题。

题目 7：向 free 传入异常指针（数组中间位置）

源代码

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *data = (int *)malloc(100 * sizeof(int));
    if (data == NULL)
    {
        perror("malloc");
        return 1;
    }

    for (int i = 0; i < 100; i++)
    {
        data[i] = i;
    }

    free(data + 50);

    printf("unreachable?\n");
    return 0;
}
```

直接运行现象

```
free(): invalid pointer
exit:134
```


valgrind 输出

```
==139509== Memcheck, a memory error detector
==139509== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==139509== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==139509== Command: ./07_badfree
==139509==
==139509== Invalid free() / delete / delete[] / realloc()
==139509==    at 0x484988F: free (in
↳ /home/amadeus/.local/valgrind/usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==139509==    by 0x10921D: main (07_badfree.c:18)
==139509==    Address 0x4a76108 is 200 bytes inside a block of size 400 alloc'd
==139509==    at 0x4846828: malloc (in
↳ /home/amadeus/.local/valgrind/usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==139509==    by 0x1091BE: main (07_badfree.c:6)
==139509==
unreachable?
==139509==
==139509== HEAP SUMMARY:
==139509==    in use at exit: 400 bytes in 1 blocks
==139509==    total heap usage: 2 allocs, 2 frees, 1,424 bytes allocated
==139509==
==139509== 400 bytes in 1 blocks are definitely lost in loss record 1 of 1
==139509==    at 0x4846828: malloc (in
↳ /home/amadeus/.local/valgrind/usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==139509==    by 0x1091BE: main (07_badfree.c:6)
==139509==
==139509== LEAK SUMMARY:
==139509==    definitely lost: 400 bytes in 1 blocks
==139509==    indirectly lost: 0 bytes in 0 blocks
==139509==    possibly lost: 0 bytes in 0 blocks
==139509==    still reachable: 0 bytes in 0 blocks
==139509==    suppressed: 0 bytes in 0 blocks
==139509==
==139509== For lists of detected and suppressed errors, rerun with: -s
==139509== ERROR SUMMARY: 2 errors from 2 contexts (suppressed: 0 from 0)
```

分析

`free` 的参数必须是动态分配函数 (`malloc/calloc/realloc`) 返回的“原始指针”(或 `NULL`)。本题错误地将 `data + 50` 传给 `free`，它指向分配块的中间位置 (偏移 50 个 `int`，即 200 字节)。分配器依赖块头信息管理内存，中间指针无法与正确的块头对应，属于严重的堆管理错误。

直接运行时，`glibc` 检测到非法指针并报 `free(): invalid pointer`，随后触发 `SIGABRT` 终止 (退出码 134)。

`valgrind` 报告 `Invalid free()`，并指出该地址位于“200 bytes inside a block of size 400”，还能定位到 `07_badfree.c:18`。另外由于没有正确释放原始 `data`，会导致 `definitely lost: 400 bytes` 的泄漏统计。

题目 8: realloc 实现 vector-like 动态数组

源代码

```
#include <stdio.h>
#include <stdlib.h>

typedef struct
{
    int *arr;
    size_t size;
    size_t cap;
} IntVec;

static void vec_init(IntVec *v)
{
    v->arr = NULL;
    v->size = 0;
    v->cap = 0;
}

static void vec_free(IntVec *v)
{
    free(v->arr);
    v->arr = NULL;
    v->size = 0;
    v->cap = 0;
}

static int vec_push_back(IntVec *v, int x)
{
    if (v->size == v->cap)
    {
        size_t new_cap = (v->cap == 0) ? 1 : (v->cap * 2);
        int *new_arr = (int *)realloc(v->arr, new_cap * sizeof(int));
        if (new_arr == NULL)
        {
            return 0;
        }
        v->arr = new_arr;
        v->cap = new_cap;
    }

    v->arr[v->size] = x;
    v->size++;
    return 1;
}
```

```

}

int main(void)
{
    IntVec v;
    vec_init(&v);

    for (int i = 0; i < 10; i++)
    {
        if (!vec_push_back(&v, i * i))
        {
            fprintf(stderr, "realloc failed\n");
            vec_free(&v);
            return 1;
        }
    }

    printf("size=%zu cap=%zu last=%d\n", v.size, v.cap, v.arr[v.size - 1]);

    vec_free(&v);
    return 0;
}

```

直接运行现象

```

size=10 cap=16 last=81
exit:0

```

valgrind 输出

```

==139666== Memcheck, a memory error detector
==139666== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==139666== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==139666== Command: ./08_vector_realloc
==139666==
size=10 cap=16 last=81
==139666==
==139666== HEAP SUMMARY:
==139666==      in use at exit: 0 bytes in 0 blocks
==139666==    total heap usage: 6 allocs, 6 frees, 1,148 bytes allocated
==139666==
==139666== All heap blocks were freed -- no leaks are possible
==139666==
==139666== For lists of detected and suppressed errors, rerun with: -s
==139666== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

性能与对比分析

该实现使用“容量不足则翻倍”的扩容策略：`cap`: 0→1→2→4→8→16…。连续插入 n 个元素时，触发 `realloc` 的次数约为 $\log_2 n$ ；虽然每次扩容可能拷贝已有元素，但总拷贝量为 $1 + 2 + 4 + \dots < 2n$ ，因此 `push_back` 的均摊时间复杂度为 $O(1)$ ，总体为 $O(n)$ 。

本程序插入 10 个元素，最终得到 `size=10 cap=16`，符合翻倍扩容的结果。

与链表对比：

- `vector-like`: 内存连续，遍历/随机访问 $O(1)$ 且缓存友好；扩容时有一次性搬移成本，但均摊插入仍为 $O(1)$ 。
- 链表: 若维护尾指针，尾插为 $O(1)$ 且不会整体搬移；但每个节点一次 `malloc/free` 带来较大常数开销，内存碎片多、缓存不友好，随机访问为 $O(n)$ 。